DBMS COURSE PROJECT · FINAL PRESENTATION

Design and Implementation of a Database Management System for

Hostel Accommodation and Student Services Management System



GROUP MEMBERS

Mohammed Amaan Khan · 25WU0101075
Arslan Alam · 25WU0101016
Jaswant Reddy · 25WU0101056

The problem with hostel registers

Today

Beds allotted twice, or to a student who already has a room

Complaint slips lost; nobody knows who fixed what, or when

Rent receipts on paper; defaulters listed by hand every month

No quick way to find a free bed in a girls' or boys' hostel

This project

Rooms, residents, requests and rent in one MySQL database

Allotment rules checked by the database: capacity, gender, one room each

Every service request tracked from Open to Resolved with a full history

Rent dues and free beds are one query away

Intended users

Students
raise requests and pay rent

Wardens
allot rooms and track dues

Service staff
resolve requests step by step

Project objectives

1. **Normalized schema**
   Store hostels, rooms, students, staff and departments in 3NF.

2. **Room allotment**
   Allot and vacate rooms, keep every stay as history.

3. **Service workflow**
   Track requests through Open, Assigned, In Progress, Resolved.

4. **Rent tracking**
   Record monthly rent and list residents with dues.

5. **Data integrity**
   Enforce rules with constraints, triggers and procedures.

10

tables in Third Normal Form

2 stored procedures

4 triggers

3 views

9 report queries

ER design: ten entities

Department

dept_id | dept_code | dept_name

Student

student_id | roll_no | gender | dept

Staff

staff_id | name | designation

Service_Type

type_name | target_days

Hostel

hostel_id | name | Boys/Girls | warden

Room

room_no | capacity | monthly_rent

Allocation

student | room | check_in | status

Service_Request

description | status | resolved_on

Request_Update

new_status | staff | remarks | time

Fee_Payment

fee_month | amount | mode

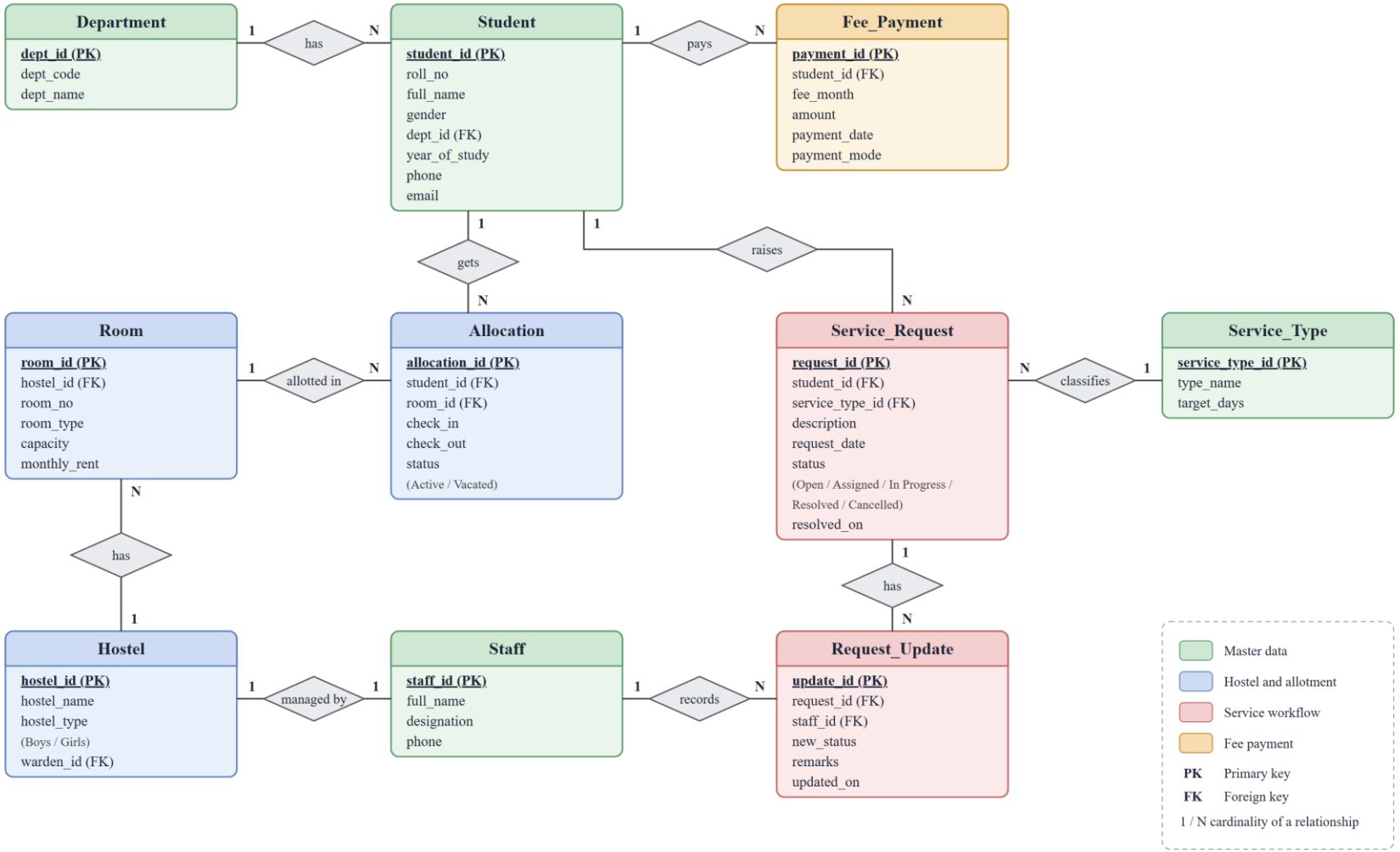
Master data

Hostel and allotment

Service workflow

Fee payment

Relationships and cardinality



1 = one, N = many. Allocation resolves the M:N relationship between Student and Room and keeps the dates of each stay.

Normalization up to 3NF

1NF

All values are atomic and every table has a primary key. Each payment and each request step is its own row.

2NF

No partial dependencies. Composite candidate keys such as (hostel_id, room_no) determine the whole row.

3NF

No transitive dependencies. Hostel names, rent and service type targets each live in one table only.

Example: rent

room_id → monthly_rent

Rent is stored once in Room. Allocation keeps only room_id, so a rent change is a single UPDATE and every resident sees it.

No update, insertion or deletion anomaly.

Service request workflow

1

Student raises

sp_raise_request checks residency and duplicates.

2

Staff assigns

Staff of the matching trade take it.

3

Staff works

Work has started; remarks are saved.

4

Staff closes

Trigger stamps resolved_on.

Status: Open → **Status: Assigned** → **Status: In Progress** → **Status: Resolved**

Enforced by the database

trg_update_before_insert allows only the next step, and nothing after Resolved or Cancelled.

trg_update_after_insert copies the status to the request. Every step is a row in request_update: staff, remarks, time.

Room allotment has its own trigger: no full room, no hostel of the other gender, one active room per student.

Database features

1	**Constraints**	PK, FK, UNIQUE (hostel_id, room_no), (student_id, fee_month); CHECK capacity 1 to 3, year 1 to 4, check_out >= check_in.
2	**Indexes**	allocation(room_id, status), allocation(student_id, status), service_request(status), fee_payment(fee_month).
3	**Views**	v_room_occupancy, v_open_requests and v_fee_summary.
4	**Procedures**	sp_allocate_room allots a bed; sp_raise_request accepts only residents and one open request per type.
5	**Triggers**	Allotment rules, the request step order and status update, and rent equal to the room's rent.
6	**Queries**	Nine report queries (Q1 to Q9): free beds, open requests, request history, dues, collection per hostel.

Live demo: database and queries

1

Create

Run hostel_management_system.sql: tables, keys, checks, indexes, views, procedures, triggers.

2

Insert

Sample data loaded: at least five records in every table, allotments made through the procedure.

3

Query

Run Q1 to Q9: joins across four tables, GROUP BY, HAVING, NOT EXISTS, views.

4

Trigger

Try to allot a full room or skip a request step and show MySQL refusing it.

Expected results with the sample data

OCCUPANCY FORMULA

Free beds = capacity − Active allocations of the room (view v_room_occupancy)

7 / 15

beds taken after the script (47%)

5

residents with October rent due (Q6)

2

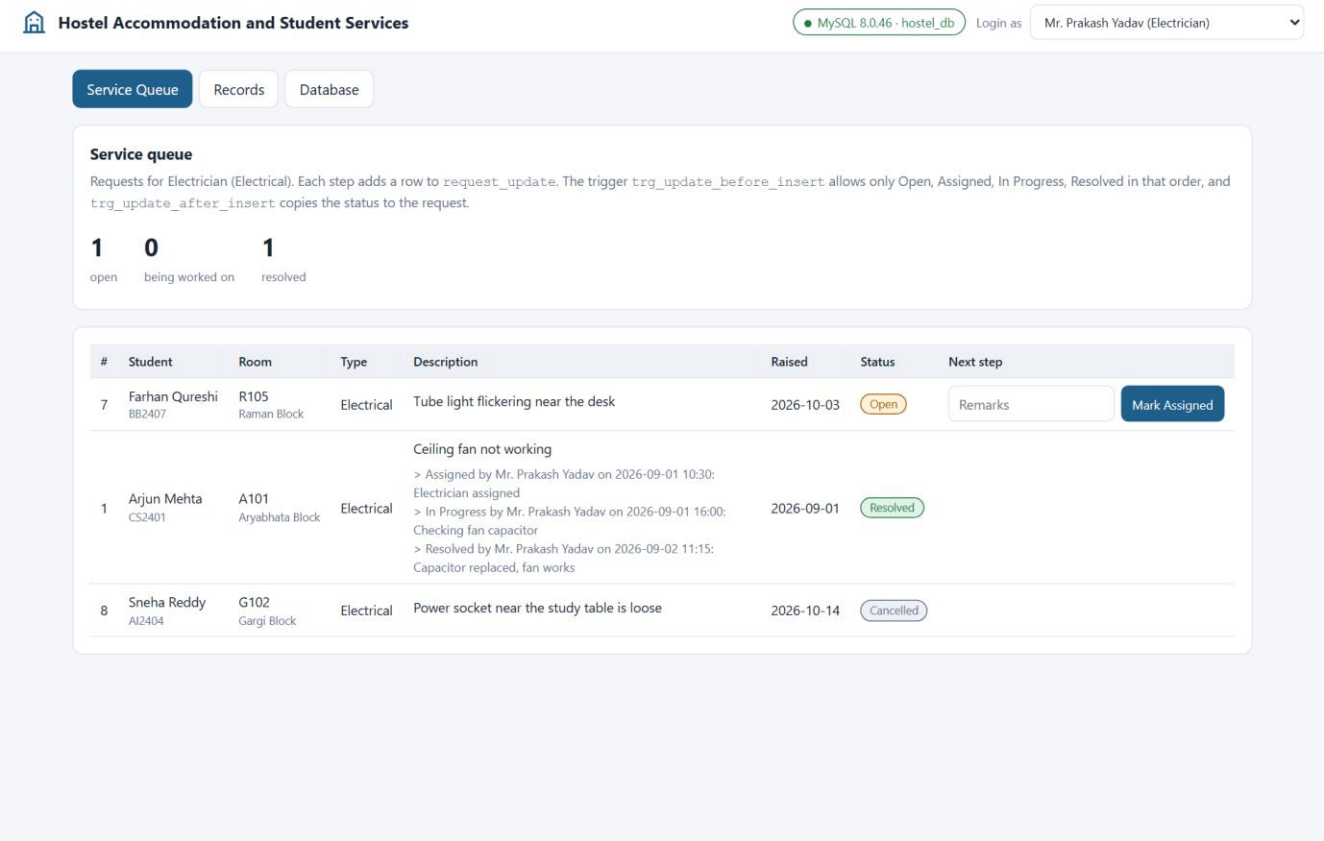
requests resolved, 3 still Open

Divya Menon is refused Aryabhata Block (boys) and G101 (full), then allotted M301: occupancy becomes 8 of 15.

Request #7 goes Open, Assigned, In Progress, Resolved; the trigger fills in resolved_on.

Rohan Das pays Rs. 4,000: refused by the trigger. He pays Rs. 4,500: accepted, and he drops off the dues list.

User interface demo



Node.js + Express + mysql2, connected to MySQL

Insert

Raise a request, allot a room, pay rent, or add a row in Records; then show the new row in MySQL.

Delete

Cancel a request or delete a row in Records; a hostel still in use is refused by the foreign key.

View

Occupancy, dues and the Database tab, matched with SELECT in MySQL.

Every page shows the SQL statement sent to MySQL.

Conclusion and future scope

CONCLUSION

Hostel rooms, allotment, service requests and rent now live in one normalized database. The rules are enforced by MySQL itself, so they hold however the data is entered.

POSSIBLE EXTENSIONS

Password login with role-based access

Email or SMS alerts on request status and dues

Online rent payment with late fees

Mess menu, mess bill and visitor log

Thank you

Questions?